

Tecnológico de Costa Rica

Introducción a la programación

(Proyecto #2)

Grupo #2

Profesor: Jeff Schmidt

Integrantes:

Ashley Rachel Mora Otárola

Victoria Sofía Fernández Martínez

Código de Estudiante:

Ashley: 2026102839

Victoria: 2026120151

Fecha de Entrega: Domingo 28 de Junio

2026

1. DESCRIPCIÓN GENERAL BATTLE CAMP

Battle Camp es un videojuego de táctica por turnos diseñado para dos jugadores. La mecánica central implementa un sistema de confrontación donde un jugador asume el rol de Defensor y la contraparte como el Atacante. El escenario de combate se despliega sobre un tipo tablero de ajedrez, cuyo núcleo contiene una base central fija que simboliza el objetivo de victoria o derrota del juego.

Las partidas se administran rigurosamente mediante un sistema secuencial de rondas. Cada ronda inicia con la Fase de Construcción, periodo exclusivo donde el Defensor posiciona, con presupuesto limitado, muros y torres con atributos especializados de combate. Posteriormente, se da paso a la Fase de Ataque, donde el Atacante moviliza estratégicamente unidades para abrir brechas en el perímetro y colapsar la resistencia de la base central del enemigo. El marco lógico establece la condición de victoria definitiva para el primer jugador que logre adjudicarse exitosamente un total de 3 rondas individuales.

2. CONFIGURACIÓN DE FACCIÓNES

El juego implementa una arquitectura visual que divide el comportamiento estético en tres facciones bien diferenciadas. Esta propiedad se modela de a través de la clase Facciones, la cual no solo altera la paleta de colores sino que además gestiona dinámicamente la reproducción musical administrada por Pygame.

```
12
13 #CLASE DE FACCIÓNES
14 class Facciones: #Molde para crear objetos tipo facción
15     def __init__(self, nombre, fondo_menu, botones, texto_boton, cancion):
16         self.nombre = nombre # guarda características individuales dentro del objeto para que no se pierdan
17         self.fondo_menu = fondo_menu
18         self.botones = botones
19         self.texto_boton = texto_boton
20         self.cancion = cancion
21
22 #3 instancias de la clase en lista que almacenan las 3 facciones disponibles
23 lista_facciones = [
24     Facciones("Medieval", "#360857", "#88743F", "white", "Medieval.mp3"),
25     Facciones("Futurista", "#0F172A", "#06B6D4", "white", "Futurista.mp3"),
26     Facciones("Zombie", "#2D3111", "#811818", "white", "Zombie.mp3")
27 ]
28
29 #Variables globales para rastrear el estado actual
30 faccion_actual = lista_facciones[0] # Define con cuál facción se empieza por defecto, empieza con medieval
31 ventana_actual = None #Para recordar cuál ventana secundaria está abierta, empieza vacía
32 musica_activa = False #Si la musica está en ON u Off
33
```

Las tres facciones corresponden a:

Medieval: Caracterizada por una atmósfera clásica mística. Configura un esquema de fondo profundo codificado en tono púrpura imperial (#360857), con botones realzados en ocre dorado (#88743F) y música ambiental ligada al archivo de audio Medieval.mp3.



Futurista: Diseñada bajo una estética de ciencia ficción ciberpunk. Establece un fondo de interfaz en azul pizarra oscuro (#0F172A), acentuaciones en botones mediante un tono cian eléctrico (#06B6D4) y una banda sonora cargada desde el archivo Futurista.mp3.



Zombie: Inspirada en un entorno postapocalíptico de supervivencia. Emplea un color de fondo verde militar oliva (#2D3111), botones contrastados en rojo sangre (#811818) y una pista musical tétrica asignada al archivo Zombie.mp3.



El juego abre de forma predeterminada con la facción Medieval. Al abrir la selección mediante el menú interactivo, la función global `cambiar_faccion` desencadena

```
#Función para cambiar el tema visual, se activa cuando el usuario elige una opción diferente en el menú de facciones
def cambiar_faccion(nombre_seleccionado):
    global faccion_actual, musica_activa # Avisar que va a cambiar las variables
    for faccion in lista_facciones: # Recorre lista de facciones una por una
        if faccion.nombre == nombre_seleccionado: # Si el nombre de la lista coincide con el seleccionado, guarda el objeto en
            faccion_actual = faccion
            break # Detiene el ciclo con break

    # Extraer y aplicar los nuevos colores
    main_frame.config(bg=faccion_actual.fondo_menu) # Redibuja visualmente el menú principal cada que se cambia de faccion
    Boton1.config(bg=faccion_actual.botones, fg=faccion_actual.texto_boton)
    Boton2.config(bg=faccion_actual.botones, fg=faccion_actual.texto_boton)
    Boton3.config(bg=faccion_actual.botones, fg=faccion_actual.texto_boton)
    Boton4.config(bg=faccion_actual.botones, fg=faccion_actual.texto_boton)
    boton_cerrar_principal.config(bg=faccion_actual.botones, fg=faccion_actual.texto_boton)
    label_faccion.config(bg=faccion_actual.fondo_menu, fg=faccion_actual.texto_boton)
```

un

barrido de reconfiguración estética instantánea sobre los widgets activos y utiliza controladores estructurados de excepciones (`try/except pygame.error`) para detener la pista musical en curso y reproducir en bucle continuo e infinito

(`play(-1)`) la nueva canción correspondiente.

```
53
54 # 2. Control de cambio de música
55 if musica_activa: # Si la musica esta activa, la cambia automáticamente
56     try:
57         pygame.mixer.music.stop() # Detiene la canción de la facción anterior
58         pygame.mixer.music.load(faccion_actual.cancion) # Carga la canción de la nueva facción
59         pygame.mixer.music.play(-1) # La reproduce en bucle infinito
60     except pygame.error: # Si al cargar el sonido hay un problema con el archivo,
61         # muestra un aviso para evitar que se cierre el juego por completo
62         print(f"No se pudo cargar el archivo de audio: {faccion_actual.cancion}")
63
64
```

3. SISTEMA DE ARCHIVOS

La gestión y almacenamiento de la información de los jugadores se realiza mediante la importación de JSON. La manipulación del sistema de archivos se delega de forma directa a la clase RegistroUsuarios, abstrayendo el acceso físico al archivo de almacenamiento secundario denominado de forma fija usuarios.json.

```
31
32 "GESTIONADOR DE TODOS LOS USUARIOS Y GUARDADO"
33 class RegistroUsuarios: #Encargada de registrar todos los usuarios y guardar sus datos
34
35     'Atributos'
36     def __init__(self):
37         self.lista_usuarios = [] #Guarda a todos los usuarios en una lista de listas
38         self.archivo = "usuario.json" #Obtiene los datos guardados de los usuarios
39         self.cargar_usuarios() #LLamada a una función que sirve para cargar los datos guardados de todos los usuarios
40
41     'Métodos'
42     def cargar_usuarios(self): #Función que sirve para buscar a un usuario en específico dentro de los archivos guardados
43         try: #El try es para buscar
44             archivo = open(self.archivo, "r") #Sirve para abrir los archivos guardados
45             datos = json.load(archivo) #el .load sirve para leer el archivo JSON e interpretarlo a un tipo de dato entendible
46             archivo.close() #Función para cerrar el archivo
47
48             for fila in datos: #El for es utilizado para recorrer la lista con información de los usuarios
49                 usuario = Usuario(
50                     fila[0], #Nombre de usuario en la posición 0
51                     fila[1], #Contraseña en posición 1
52                     fila[2], #Victorias como atacante en posición 2
53                     fila[3] #Victorias como defensor en posición 3
54                 )
55
56                 self.lista_usuarios.append(usuario) #El .append concatena los usuarios en una matriz
57         except: #Significa que si no hay ningún usuario registrado, entonces se devuelve una lista vacía
58             self.lista_usuarios = [] #Muestra lista vacía sin usuarios
59
```

Al arrancar el hilo principal de ejecución, el sistema efectúa una lectura automática empleando la primitiva estructurada `json.load()`, deserializando los arreglos anidados y poblando una lista en memoria dinámica compuesta por instancias de objetos lógicos de tipo `Usuario`. Cada registro almacena de forma estricta los siguientes atributos primitivos:

usuario: Cadena de caracteres alfanumérica única que actúa como identificador.

contrasena: Clave de autenticación resguardada y validada en texto plano.

victorias_atacante: Contador entero acumulativo que computa las rondas ganadas en rol ofensivo.

victorias_defensor: Contador entero acumulativo que computa las rondas ganadas en rol defensivo.

```
59
60 def guardar_usuarios(self): #Función que guarda los datos de los usuarios
61     datos = [] #Lista vacía para poder agregar nuevos datos
62
63     for usuario in self.lista_usuarios: #recorre la lista
64         fila = [
65             usuario.obtener_nombre(), #Nombre obtenido desde la clase Usuario
66             usuario.obtener_contrasena(), #Contraseña obtenida desde la clase Usuario
67             usuario.obtener_victorias_atacante(), #Victorias actuales como defensor obtenidas desde la clase Usuario
68             usuario.obtener_victorias_defensor() #Victorias actuales como defensor obtenidas desde la clase Usuario
69         ]
70
71         datos.append(fila) #Concatenación de datos
72
73     archivo = open(self.archivo, "w") #Abre el archivo y se escribe dentro de él
74     json.dump(datos, archivo) #El json.dump sirve para convertir datos de Python a datos de formato JSON
75
76     archivo.close() #Cierra el archivo
```

Cada vez que ocurre una mutación de estado en el sistema (como el registro exitoso de un nuevo perfil mediante la función crear_usuario), se invoca de manera imperativa el método guardar_usuarios.

```
77
78 def crear_usuario(self, nombre, contrasena): #Función para condiciones de crear nuevos usuarios
79     for usuario in self.lista_usuarios: #Recorre la lista
80         if usuario.obtener_nombre() == nombre:
81             return False
82
83     nuevo = Usuario(nombre, contrasena) #Se crea nuevo usuario
84     self.lista_usuarios.append(nuevo) #Se concatena la información
85     self.guardar_usuarios() #Se llama a la función guardar_usuarios para guardar el nuevo usuario
86
87     return True
88
89
90 def iniciar_sesion(self, nombre, contrasena):
91     for usuario in self.lista_usuarios:
92         if usuario.obtener_nombre() == nombre:
93             if usuario.obtener_contrasena() == contrasena:
94                 return usuario
95     return None
```

Esta subrutina abre un canal de escritura (open(..., "w")), serializa las colecciones de datos a través de json.dump() y libera el descriptor del sistema operativo invocando explícitamente al método close(), garantizando la consistencia ante fallos imprevistos de la aplicación.

4. ESTRUCTURAS DE DATOS UTILIZADAS

Matrices: Corresponde al núcleo lógico del mapa de juego. Implementado en la clase

TableroJuego mediante la variable interna

self.matriz_logica = [[None for _ in range(10)] for _ in range(10)]. Estructura una

cuadrícula exacta de 10

filas por 10

columnas,

inicializada con

referencias vacías.

Cuando una

estructura de defensa

es adquirida, la celda

```
5
6 class TableroJuego: # Clase encargada de construir y gestionar el mapa interactivo de casillas
7
8     def __init__(self, ventana_contenedor, faccion_defensor, faccion_atacante, defensor_logico):
9         self.contenedor = ventana_contenedor
10        self.faccion_defensor = faccion_defensor # Objeto de la facción elegida por el jugador 1 (Defensor)
11        self.faccion_atacante = faccion_atacante # Objeto de la facción elegida por el jugador 2 (Atacante)
12        self.defensor = defensor_logico # Conexión lógica con Torres.py para manejar dinero y compras
13
14        # Matriz lógica de 10x10 que inicia vacía (None) para registrar las estructuras
15        self.matriz_logica = [[None for _ in range(10)] for _ in range(10)]
16        self.casillas = [] # Guardará las referencias visuales de cada botón tk.Button
17
18        # Obtiene los colores de fondo del mapa basados en el defensor
19        self.color1, self.color2 = self.obtener_colores_faccion()
20
21        # --- ATRIBUTOS INDIVIDUALES PARA LAS IMÁGENES PIXEL ART (SIN DICCIONARIOS) ---
22        self.imagen_torre_basica = None
23        self.imagen_torre_magica = None
24        self.imagen_torre_pesada = None
25        self.imagen_muro = None
26
27        # Método interno para cargar los archivos físicos .png antes de pintar el tablero
28        self.cargar_recursos_imagenes()
29
30        # Ejecuta la construcción visual de la cuadrícula
31        self.generar_tablero()
```

correspondiente conmuta su estado de None a la referencia directa del objeto lógico de la

torre instalada, permitiendo búsquedas posicionales de orden de complejidad constante.

Listas Dinámicas: Utilizadas en múltiples veces en el software. El gestor de cuentas

encapsula la variable

self.lista_usuarios = []

para mantener el índice

activo de jugadores. De

igual forma, la clase

```
32 "GESTIONADOR DE TODOS LOS USUARIOS Y GUARDADO"
33 class RegistroUsuarios: #Encargada de registrar
34
35     'Atributos'
36     def __init__(self):
37         self.lista_usuarios = [] #Guarda a todos
38         self.archivo = "usuario.json" #Obtiene l
39         self.cargar_usuarios() #LLamada a una fu
40
```

Defensor administra la estructura lineal self.torres = [] para indexar el inventario

dinámico de mecanismos balísticos contruidos por ronda.


```

109
110 "JUGADOR DE DEFENSA"
111 class Defensor: #Clase que representa a quien defiende
112     def __init__(self):
113         self.dinero = 300 #Dinero inicial
114         self.torres = [] #Lista donde se guardarán las tor
115

```

5. UNIDADES Y TORRES

El sistema de combate y el balance táctico del juego en el módulo Torres.py. El diseño aplica herencia simple, donde la clase denominada Torres provee los atributos transversales e interfaces base, de la cual se derivan tres especializaciones concretas de defensa:

Tipo de estructura	Costo en monedas	Puntos de vida	Daño base	Alcance en casillas	Habilidad
Torre básica	50	100	15	2	Reduce el tiempo de recarga global de disparo.
Torre pesada	135	200	40	2	Duplica el daño infligido (D = Daño imes 2).
Torre mágica	75	75	10	3	Altera el estado del enemigo a congelado (True).

Para validar si una unidad atacante se encuentra dentro del rango de acción de una torre defensiva el método matemático `verificación_rango` calcula la distancia basándose en los ejes de la cuadrícula de la siguiente forma:

```
31
32     def verificación_rango(self, fila_torre, columna_torre, fila_unidad, columna_unidad):
33         distancia = abs(fila_torre - fila_unidad) + abs(columna_torre - columna_unidad) #0
34
35         if distancia <= self.alcance: #Condición de que debe ser una distancia menor o ig
36             return True #La distancia es menor o igual al alcance de la torre, así que se
37         return False #La distancia es mayor, no cumple la condición
38
```

El algoritmo calcula las diferencias de coordenadas utilizando la función de valor absoluto `abs()` para suprimir signos negativos. Si el resultado entero de la ecuación es estrictamente menor o igual que el atributo de cobertura (`self.alcance`) de la torre evaluada, el sistema valida el emparejamiento táctico, retornando un valor booleano positivo que habilita el ciclo de ataque.

6. INSTRUCCIONES

Para poner en marcha el videojuego Battle Camp de forma correcta se debe seguir la siguiente secuencia procedimental:

- Asegúrese de tener instalada una versión estable de Python 3.8 o superior en el sistema operativo.
- Abra su terminal y ejecute los comandos necesarios para instalar las bibliotecas de procesamiento de imágenes y soporte de audio que no vienen en el núcleo de Python con: `pip install Pillow pygame`
- Es mandatorio ubicar todos los módulos fuente del código (`Interfaz.py`, `ComponentesJuego.py`, `Torres.py`, `RegistroyCuenta.py`, y `Puntajes.py`)

exactamente en la misma carpeta raíz. Asimismo, los recursos multimedia de audio (Medieval.mp3, Futurista.mp3, Zombie.mp3) y los archivos de imagen pixel art en formato .png referenciados por el tablero deben residir en dicho directorio.

- Desde la consola de comandos posicionada en la ruta compartida, inicialice el proceso principal ejecutando el archivo que administra la interfaz base:
`python Interfaz.py`.

7. CONCLUSIONES INDIVIDUALES

1. Victoria Fernández

Trabajar en la interfaz gráfica de **Battle Camp** me permitió entender lo importante que es el manejo de recursos multimedia para que el juego se sienta fluido y no se quede pegado. Al diseñar el tablero en Tkinter, tuve que implementar la librería PIL para hacer una precarga y redimensionamiento único de los sprites directamente en la memoria caché, evitando que el sistema colapsara o tuviera caídas de rendimiento cada vez que los jugadores interactuaban con las casillas. Además, fue un reto muy interesante programar el cambio estético de las facciones junto con la música de fondo usando Pygame, donde el uso de controladores globales y bloques estructurados try/except me ayudó a garantizar que, si un archivo de audio llega a faltar, el juego lo controle de manera limpia en lugar de cerrarse inesperadamente a mitad de una partida.

2. Ashley Mora

Por mi parte, el desarrollo de la lógica de Battle Camp me enseñó cómo poner en práctica las ventajas de la Programación Orientada a Objetos para resolver mecánicas de juego complejas. Al estructurar el módulo Torres.py, logré centralizar las reglas de combate y el cálculo de rangos con una clase base, haciendo que las torres especializadas heredaran estos comportamientos y solo tuvieran que ejecutar sus habilidades individuales, sin necesidad de duplicar código. Asimismo, diseñar el sistema de persistencia con la librería json fue un gran aprendizaje de lógica, ya que logré que los datos de los perfiles y el ranking de victorias se leyeran y actualizaran de forma transparente en el archivo usuarios.json, manteniendo un estado consistente cada vez que se termina una ronda.

